

LING 510: Week 4 review

October 2, 2026

Shaunak Phadnis

sphadnis@umass.edu

Office hours: W 11–1PM, N421H | [Zoom](#)



Notes: Truth table convention, Use [this tool](#) to write your proofs in word/doc.

1 Types

Different meanings in natural language pattern the same way and can be grouped together. We can label with **types**. We have two basic types in the system.

- Entites ($\llbracket \text{Clark} \rrbracket$, $\llbracket \text{Bruce} \rrbracket$, $\llbracket \text{Diana} \rrbracket$...) are type e ;
- Truth values are type t .

Types allow us to define *domains*. They are needed because the composition process involves **function** application.

- $D_e = \{x : x \text{ is of type } e\} = \{\text{Clark, Bruce, Diana, Taro ...}\}$
- $D_t = \{x : x \text{ is of type } t\} = \{T, F\}$

Functions map members of its domain to the members of its codomain. All functions group together under this property. We can give them a type too! Since functions come in various shapes and sizes, let's provide a general notation for **functional** types (i.e., types of functions).

Functional types

- If α and β are any types, then $\langle \alpha, \beta \rangle$ is the type of a function $f : D_\alpha \rightarrow D_\beta$.
- Note that α and β themselves can be functional types too!

We can use types to specify domains of the variables appearing in our lambda expressions. We have seen that the meaning of transitive predicates like *smokes* is a function that takes entities and maps them to truth values. Its type is $\langle e, t \rangle$.

$$\llbracket \text{smokes} \rrbracket = [\lambda x : x \in D_e. \text{True iff } x \text{ smokes}] = [\lambda x_e. \text{True iff } x \text{ smokes}] = \begin{bmatrix} \text{Clark} \rightarrow F \\ \text{Bruce} \rightarrow T \\ \text{Diana} \rightarrow F \\ \vdots \end{bmatrix}$$

We also saw that the meaning of transitive predicates like *eats* is a function that takes entities and returns a function that takes entities to truth values. So their type is $\langle e, \langle e, t \rangle \rangle$.

$$\llbracket \text{eats} \rrbracket = [\lambda x : x \in D_e. [\lambda y : y \in D_e. \text{T iff } y \text{ eats } x]] = [\lambda x_e. [\lambda y_e. \text{T iff } y \text{ eats } x]] =$$

$$\left[\begin{array}{l} \text{cookie} \rightarrow \left[\begin{array}{l} \text{Clark} \rightarrow F \\ \text{Bruce} \rightarrow T \\ \text{Diana} \rightarrow F \\ \vdots \end{array} \right] \\ \text{chips} \rightarrow \left[\begin{array}{l} \text{Clark} \rightarrow T \\ \text{Bruce} \rightarrow F \\ \text{Diana} \rightarrow T \\ \vdots \end{array} \right] \\ \text{natto} \rightarrow \left[\begin{array}{l} \text{Clark} \rightarrow F \\ \text{Bruce} \rightarrow F \\ \text{Diana} \rightarrow F \\ \vdots \end{array} \right] \\ \vdots \end{array} \right]$$

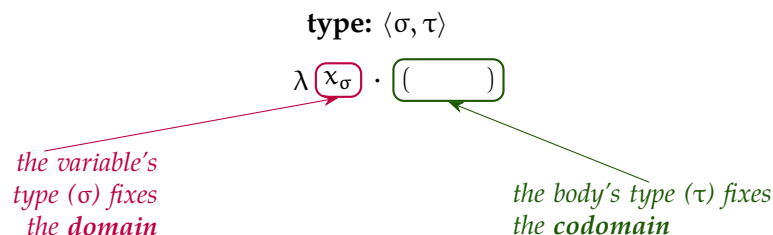
To summarize: All expressions denoting an individual are the same type. So are all expressions denoting a proposition, an intransitive predicate, etc.

Type inventory

Our type inventory so far consists of the following types. Can you read out the functions they represent? What kind of expressions do they correspond to?

- e
- t
- $\langle e, t \rangle$
- $\langle t, t \rangle$
- $\langle t, \langle t, t \rangle \rangle$

All our computations are going to end up returning type t in the end. t is a terminal type. nothing else is a terminal type since we are computing meanings of sentences i.e., truth conditions. Introducing types into our system allows us to read off of the lambda expression what kind of function it represents – we do this by looking at the types of variables and the type of their value description. Recall that the anatomy of lambda expressions looks as follows:



The schema: A function from $\langle \text{insert domain} \rangle$ to $\langle \text{insert codomain} \rangle$.

The domain itself may be a complex type or the body may itself contain another lambda expression (yielding a complex type)– repeat this process on it until you reach the terminal type. This means that now a lambda expression's domain or codomain may consist of functions. Which means we can have lambda expressions who take functions as their input argument. This is super useful for modeling natural language meaning.

Exercise

What is the type of each of the following lambda expressions? Respect the brackets!

1. $[\lambda x_e. \text{True iff } x \text{ knits}]$
2. $[\lambda f_{\langle e, t \rangle}. \text{True}]$
3. $[\lambda f_{\langle e, t \rangle}. \text{True iff } f(b) = \text{True}]$
4. $[\lambda f_{\langle e, t \rangle}. [\lambda g_{\langle e, t \rangle}. \text{True iff } f(b) = g(b) = \text{True}]]$
5. $[\lambda f_{\langle e, t \rangle}. [\lambda g_{\langle e, t \rangle}. [\lambda x_e. \text{True iff } f(x) = g(x) = \text{True}]]]$
6. $[\lambda x_e. [\lambda f_{\langle e, t \rangle}. [\lambda g_{\langle e, t \rangle}. \text{True iff } f(x) = g(x) = \text{True}]]]$

2 Type driven interpretation

Our system has the following building blocks.

1. **Syntax** feeds input in the form of trees (i.e. syntactic structures).
2. **Semantics** provides denotations for the input using the lexicon and the interpretation rules.
3. **Interpretation rules** tell you how to travel from Syntax to Semantics.¹ So far we have three rules.

- (a) **Terminal nodes (TN):** look up the dictionary (i.e. lexicon) if you want to know the meaning of a lexical item.
- (b) **Non-branching nodes (NN):** single parents (i.e. non-branching nodes) pass upwards the type (and meaning) of their daughter.
- (c) **Function application (FA):** If I, a type $\langle \sigma, \tau \rangle$ node, have a sister who is part of my diet (i.e., a type σ node), I eat her and return τ to our parent.

Notice that choosing how to apply the rule of function application depends entirely on the meaning of the daughter nodes. It makes no reference to order. Formal definition is as follows,

(1) Function application

If α is a binary branching node, β, γ are its daughter nodes, and β is a function that can take γ as an argument, then $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket (\llbracket \gamma \rrbracket)$.

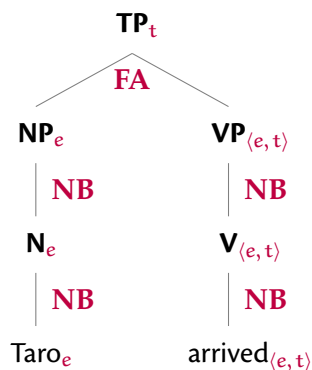
The rule of function application *does not* care about

- The order of the daughters;
- Their syntactic category;
- The details about their meaning.

¹You'll notice that our system can only interpret unary or binary branching trees!

We can annotate trees for the types of their nodes and the rules that apply at the nodes as follows.

(2) Taro arrived.



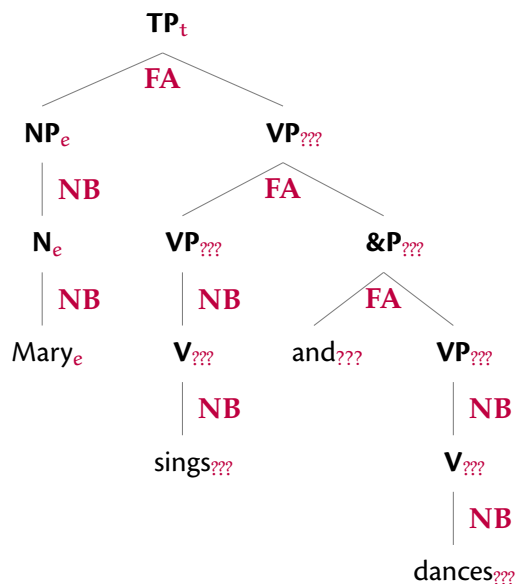
What function application does care about is whether one daughter behaves as a function that takes the other as an argument. We can read this off of the types!

☞ If β is type $\langle a, b \rangle$ and γ is type a , then β can take γ as its argument and their parent will have type b .

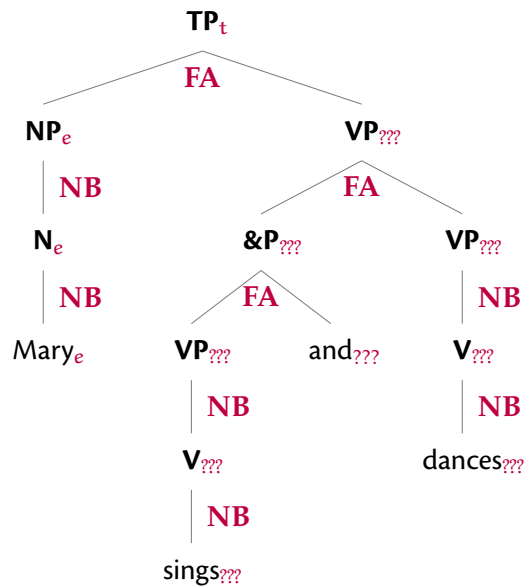
This allows us to predict the right types of constituents in a structure if we are not sure what the types are! Since we are at least sure of e and t type constituents. It can help to work from top to bottom as well.

In class we saw that natural language conjunction is of type $\langle t, \langle t, t \rangle \rangle$. We saw this for sentence level conjunction. But we can also conjoin VPs! Can you fill out the types and guess the type of VP-level conjunction for (3)?

(3) Mary sings and dances.

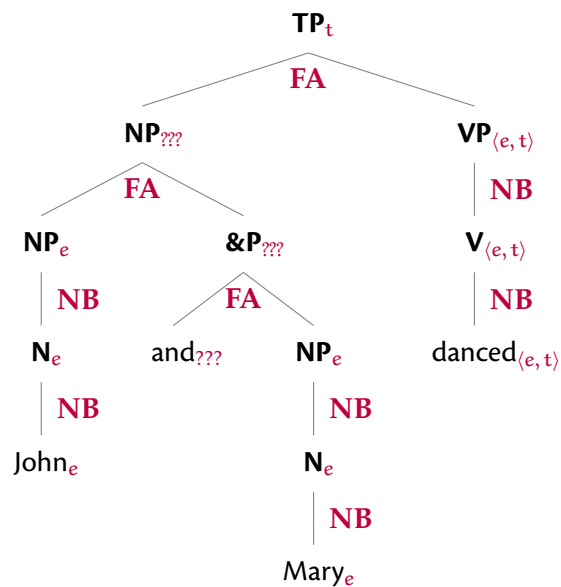


Would the type change if we changed the constituency of **&P** as in the next tree?



We can also conjoin DPs! what type do we predict for conjunction in this case?

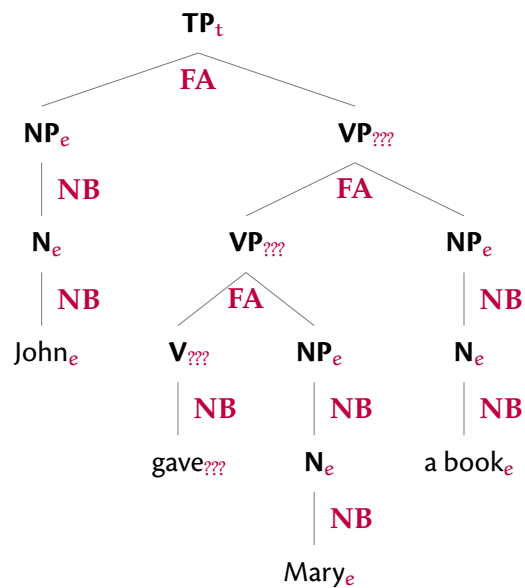
- (4) John and Mary danced.



Whoa! We just found multiple types for conjunction! Does this mean there are multiple entries of conjunction in natural language? What other constituents in English can be conjoined?

English also has ditransitive predicates. These predicates take three arguments. What type do we predict for give in (5)?

(5) John gave Mary a book.



Exercise

Can you guess the types of each function below and then perform reduction for each expression? Remember to mind the brackets!

1. $[\lambda x_e. [\lambda y_e. T \text{ iff } x \text{ saw } y]](m)$
2. $[\lambda x_e. [\lambda y_e. T \text{ iff } x \text{ saw } y]](b)(m)$
3. $[\lambda x_e. [\lambda y_e. T \text{ iff } x \text{ saw } y]](b)(m)$
4. $[\lambda f_{\langle e, t \rangle}. T \text{ iff } f(b) = T][[\lambda x_e. T \text{ iff } x \text{ knits}]]$